LAB 4

XSS, Session Management & Authentication Resilience Assessment

DVWA and OWASP Mutillidae II — Isolated Kali Linux VM (VMware NAT)

Student Name	Daniyal Ahmed
Registration Number	C11/26/EHIT/17331
Cohort	Ethical Hacking Internship Training
Course	Offensive Security Operations I (CIP-A104)
Submission Date	2 September 2026
Assessment	Lab 4

Authorised training assessment performed exclusively against locally installed, intentionally vulnerable applications inside an isolated VM. No public exposure. All evidence limited to benign markers and learner-owned test accounts.

Table of Contents

1. Scope & Instructor Briefing Summary
2. Phase 1 — XSS Baseline (Input/Output Mapping)
3. Phase 2 — Reflected/DOM XSS Proof (Harmless Marker)
4. Phase 3 — Stored XSS Cross-Session Persistence Proof
5. Phase 4 — XSS Secure-Mode Retest
6. Phase 5 — Session Lifecycle
7. Phase 6 — Cookie Attribute Table
8. Phase 7 — Authentication Resilience (5-Attempt Test)
9. Phase 8 — Comparative Analysis
10. Analysis Questions
11. Remediation Table
12. Evidence Register
13. Evidence-Minimisation Statement
14. Cleanup & Restoration Checklist
15. Conclusion

1. Scope & Instructor Briefing Summary

This report documents Lab 4 of the CIP-A104 Offensive Security Operations I course, assessing reflected, stored and DOM-based Cross-Site Scripting (XSS); session cookie behaviour including rotation and logout invalidation; and a tightly bounded authentication-resilience test (maximum five failed attempts against one dedicated account per target), performed exclusively against a locally hosted DVWA instance and OWASP Mutillidae II v2.12.7 instance inside an isolated Kali Linux VM. No cookie theft, session hijacking, phishing overlays, credential stuffing or password spraying was performed, in line with the assessment's prohibited-actions list. The nine required hands-on phases were completed in sequence: XSS baseline mapping, reflected/DOM proof-of-concept, stored XSS cross-session persistence, secure-mode retesting, session lifecycle capture, cookie attribute review, authentication resilience testing, comparative analysis, and remediation with cleanup.

2. Phase 1 — XSS Baseline (Input/Output Mapping)

Before any payload was injected, each application's input sources, output sinks and rendering context were mapped across all six target vectors — Reflected, Stored and DOM XSS, on both DVWA and Mutillidae — by submitting a plain, non-malicious test value and inspecting exactly where it reappeared in the page and in what form (raw HTML text, an attribute, or written into the DOM by client-side script).

Target	Parameter	Method	Sink Type
DVWA Reflected	name	GET	Raw HTML text (<pre>)
DVWA Stored	txtName / mtxMessage	POST	Raw HTML text (guestbook div)
DVWA DOM	default	GET (client-read)	document.write() -> <option>
Mutillidae Reflected	target_host	POST	Raw HTML text (results panel)
Mutillidae Stored	blog_entry	POST	Raw HTML text (<td>)
Mutillidae DOM (HTML5 Storage)	DOMStorageKey / Item	Client-side storage	Safe text render (non-exploitable)

DVWA Reflected XSS

The "What's your name?" field submits via GET as the **name** parameter. A baseline value ("daniyal") was submitted and reflected verbatim inside a <pre> tag directly below the form, confirmed by inspecting both the rendered output and the underlying HTML.

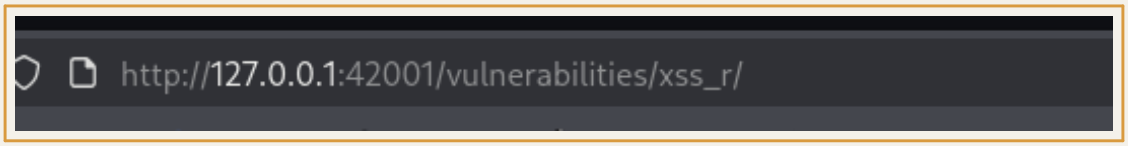


Fig 1.1 — Baseline URL, clean state

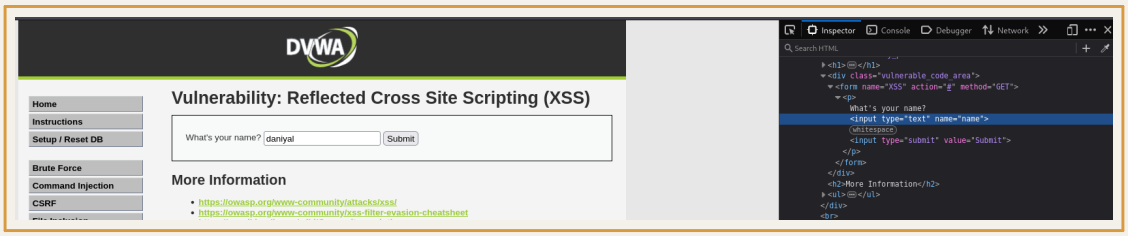


Fig 1.2 — Inspector confirming name= input attribute

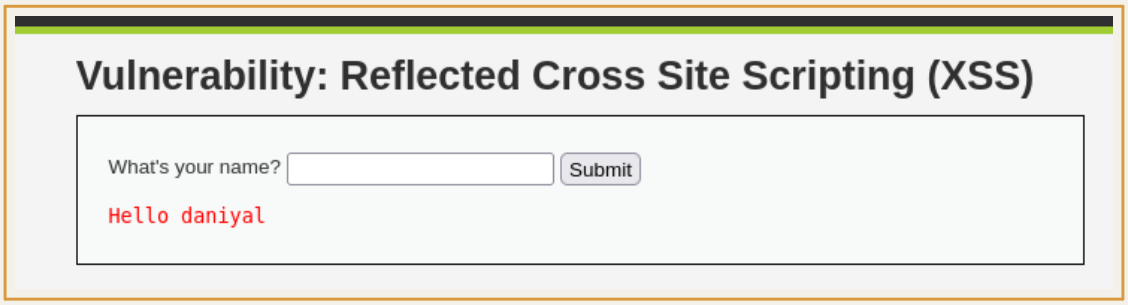


Fig 1.3 — Reflection rendered: "Hello daniyal"

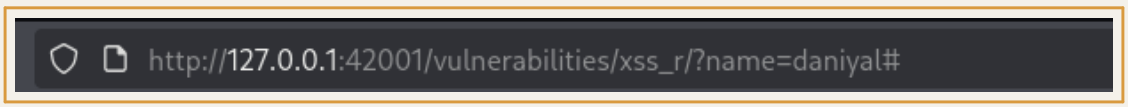


Fig 1.4 — URL confirming GET parameter name=daniyal

DVWA Stored XSS

The guestbook form submits two POST fields, **txtName** and **mtxMessage**. A baseline entry was signed and confirmed to persist after a page reload, with the rendered comment sitting inside a plain as raw HTML text — the defining sink for a persistent, database-backed vulnerability.

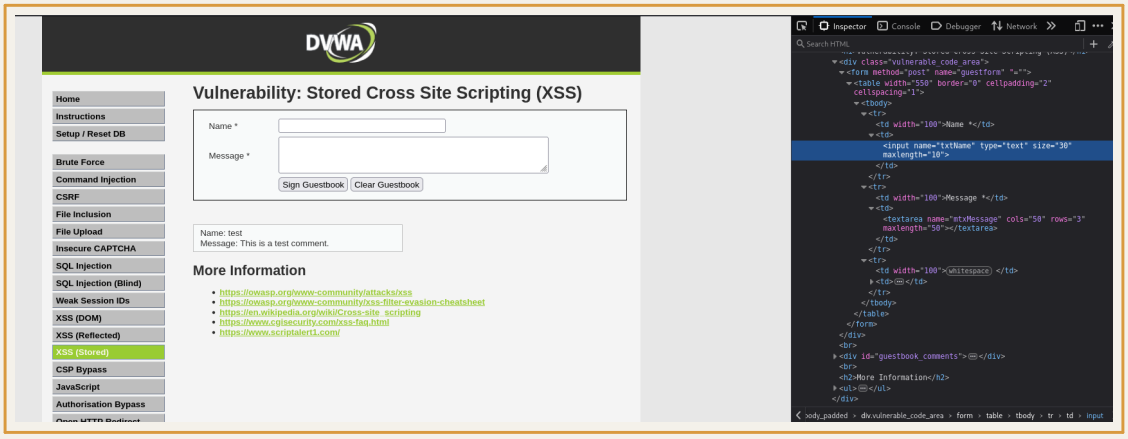


Fig 1.5 — Inspector: txtName / mtxMessage fields, method=post

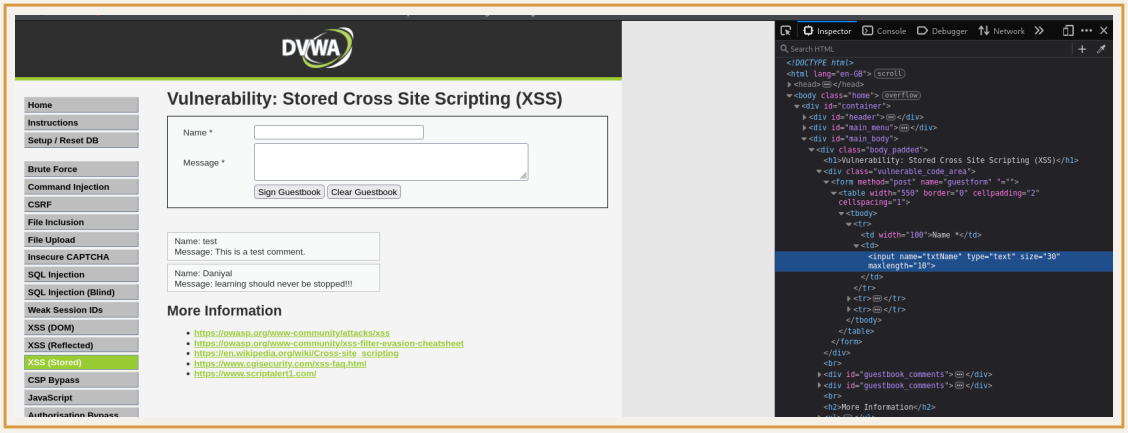


Fig 1.6 — Rendered guestbook entry

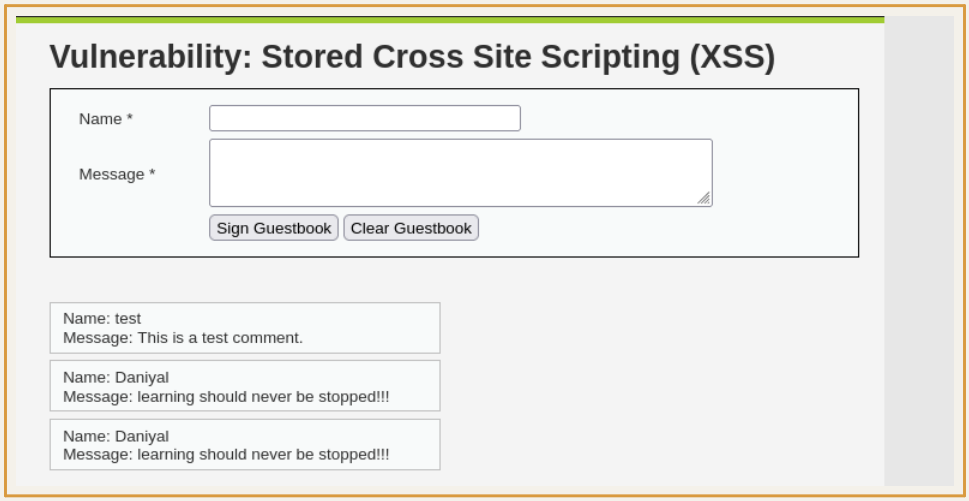


Fig 1.7 — Persistence confirmed after reload



Fig 1.8 — Inspector: sink inside guestbook_comments div

DVWA DOM XSS

This module reads a **default** value directly from document.location.href and writes it into a <select> dropdown via document.write() — entirely client-side, with no server involvement in processing this parameter.



Fig 1.9 — Page source: document.location / document.write sink

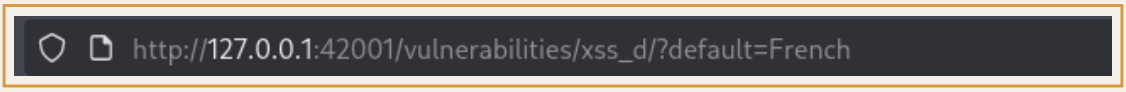


Fig 1.10 — URL confirming default= parameter

Mutillidae Reflected XSS (DNS Lookup)

The DNS Lookup page submits a **target_host** field via POST. Because the form uses POST rather than GET, the submitted value does not appear in the URL — a useful contrast point against DVWA's GET-based reflected module. The value is echoed inside the "Results for..." output panel.

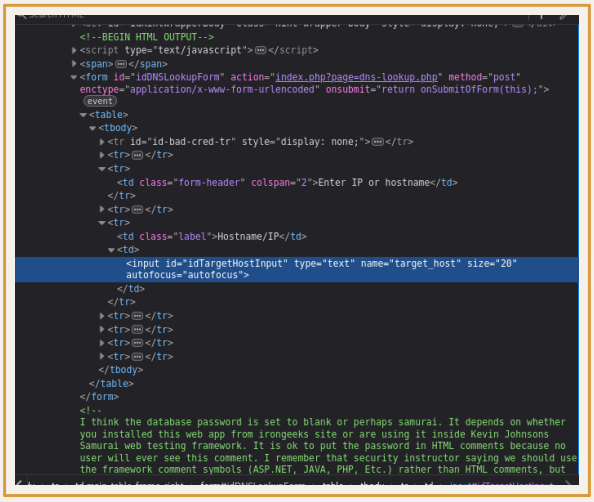


Fig 1.11 — Inspector: target_host field, method=post

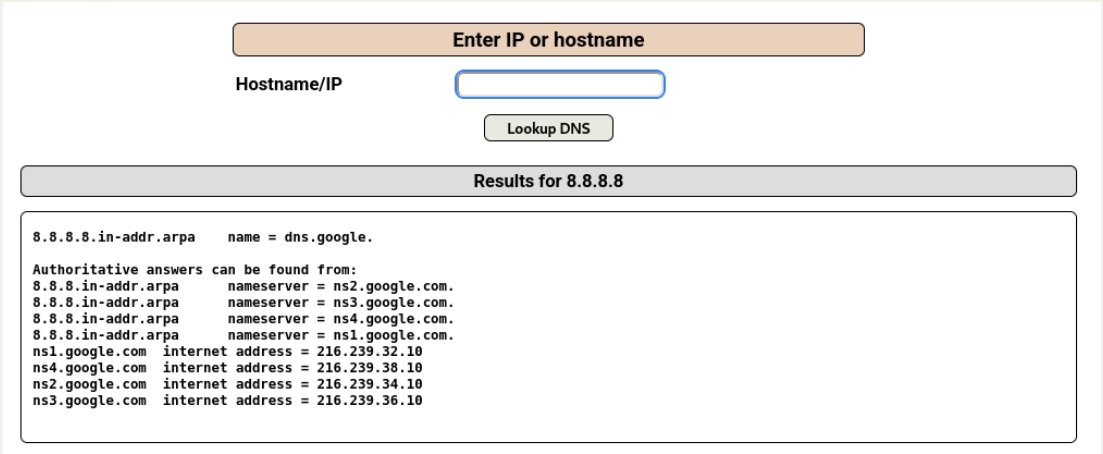


Fig 1.12 — Rendered DNS lookup results reflecting input

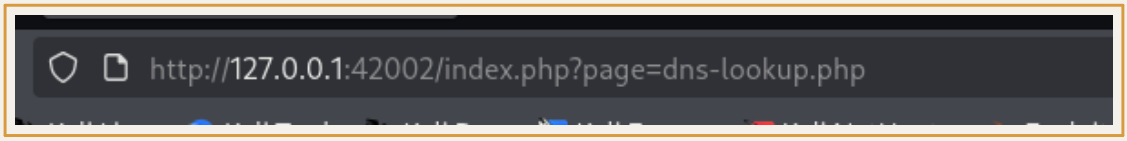


Fig 1.13 — Page URL (POST — parameter not shown in URL)

Mutillidae Stored XSS (Blog)

The "Add to your Blog" form submits a single POST field, **blog_entry**. Submitted entries render inside a raw-text <td> in the blog results table and persist across reloads, confirmed by watching the entry count grow with each genuine submission.

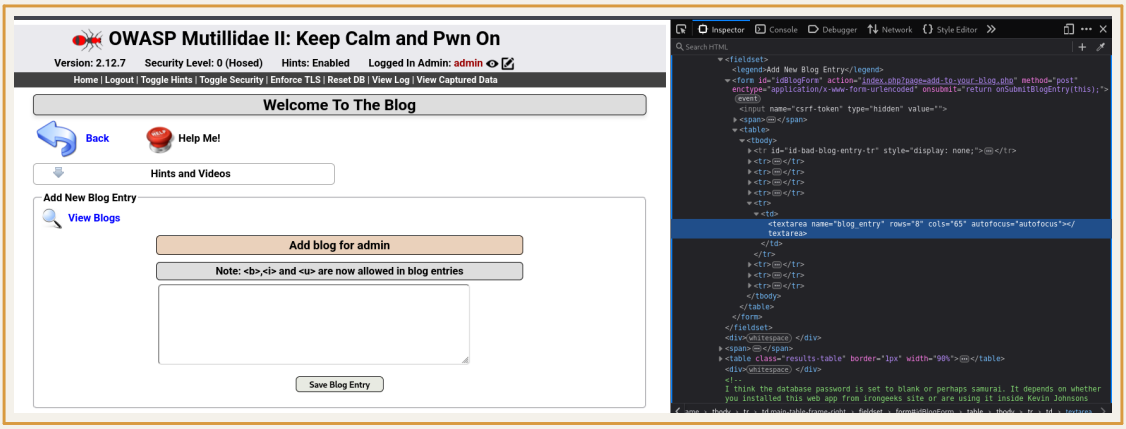


Fig 1.14 — Blog form Inspector: blog_entry field, method=post

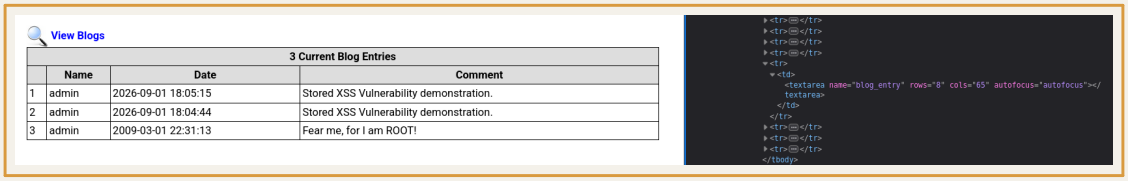


Fig 1.15 — Rendered blog entries with field Inspector

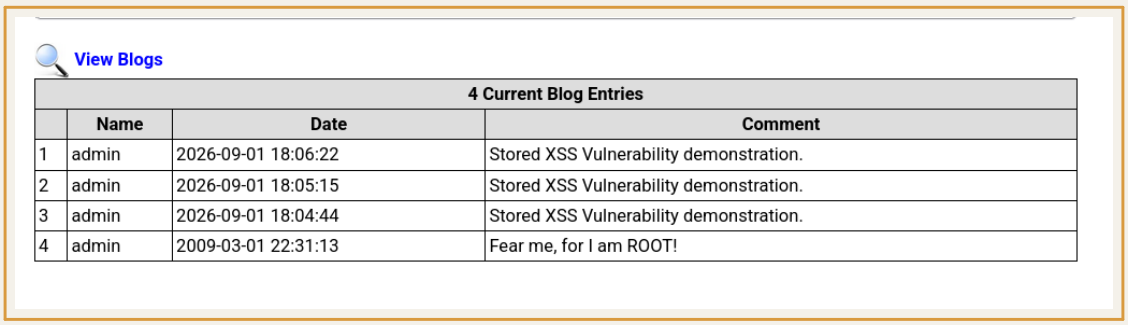


Fig 1.16 — Entry count confirming server-side persistence

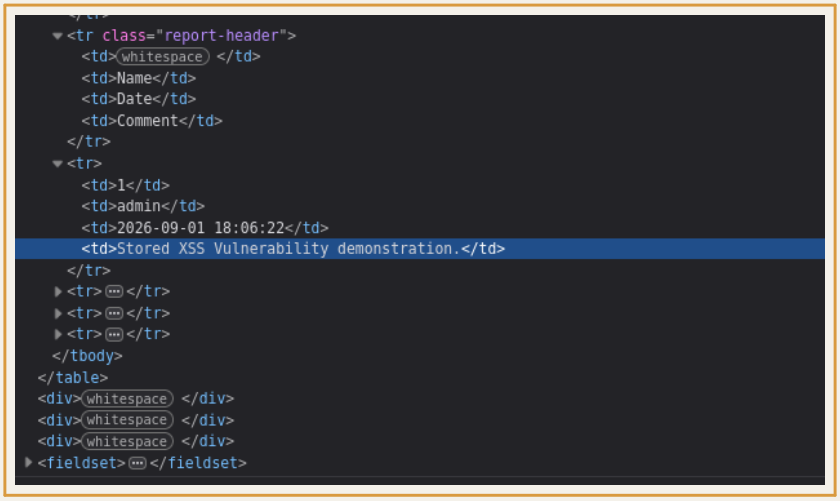


Fig 1.17 — Inspector: sink is a raw-text <td>

Mutillidae DOM XSS (HTML5 Storage)

This page reads/writes browser **sessionStorage/localStorage** via **DOMStorageKey** and **DOMStorageItem** fields and renders the stored value inside a client-rendered table row — a client-side-only sink, identical in principle to DVWA's DOM module but using a different storage API.

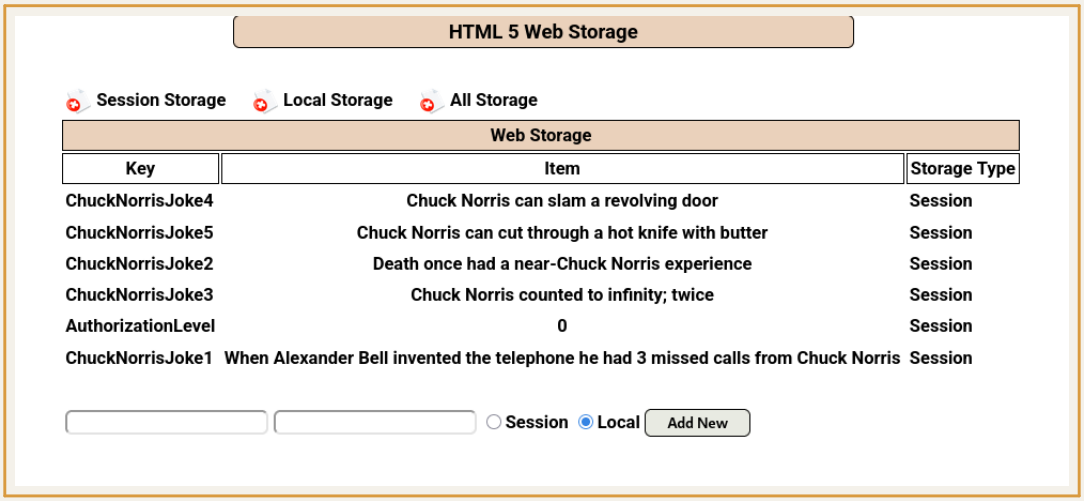


Fig 1.18 — Page source: form fields for storage key/item

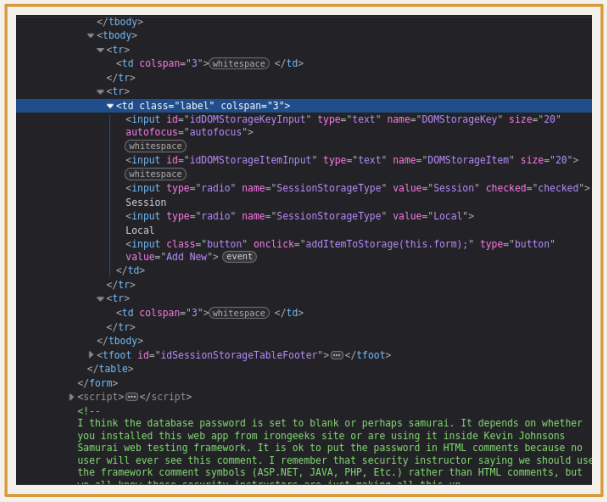


Fig 1.19 — Inspector: DOMStorageKey / DOMStorageItem inputs

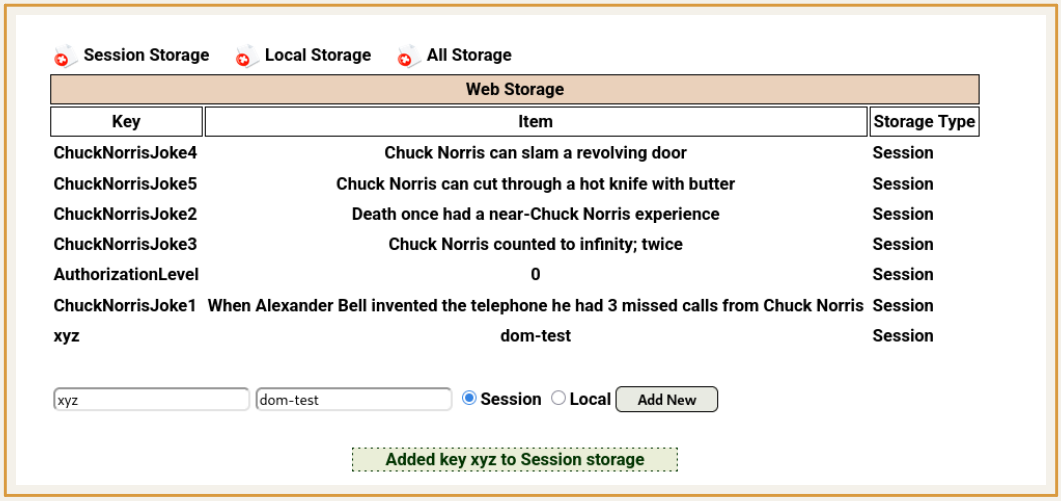


Fig 1.20 — Test entry (xyz / dom-test) added to Session Storage

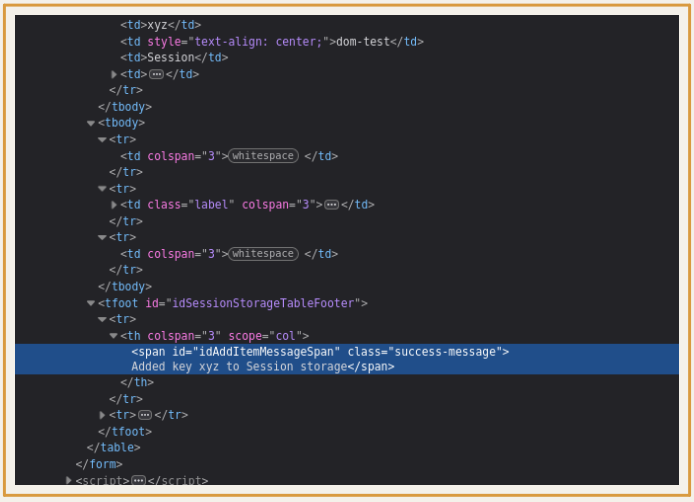


Fig 1.21 — Inspector: value rendered inside storage table <td>

3. Phase 2 — Reflected/DOM XSS Proof (Harmless Marker)

A harmless, unambiguous visual marker (<script>alert('XSS-Test')</script>) was submitted to each of the four client-executable vectors identified in Phase 1. A JavaScript alert box is sufficient proof of browser-side execution because it demonstrates the browser parsed attacker-controlled input as active, executable script rather than inert text — with zero residual effect on the target application.

DVWA Reflected XSS — Marker Fired

The marker was entered directly into the name field. On submission the browser executed the script immediately, and the page source confirmed the <script> tag was returned completely unescaped inside the <pre> sink identified in Phase 1.

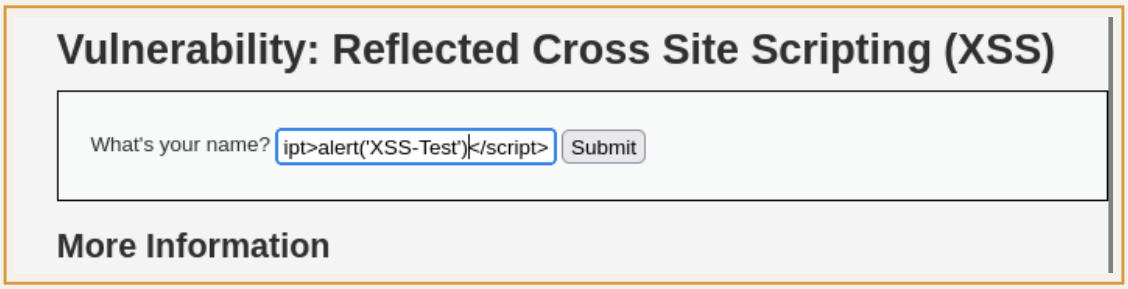


Fig 2.1 — Payload entered in input field

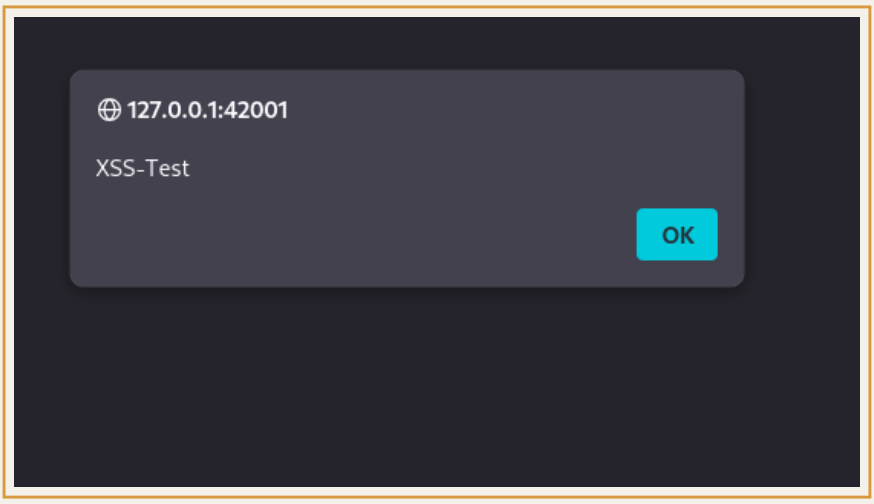


Fig 2.2 — Alert executed: "XSS-Test"

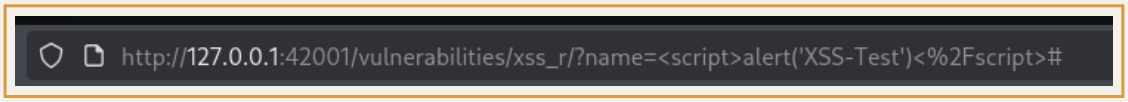


Fig 2.3 — URL showing URL-encoded payload

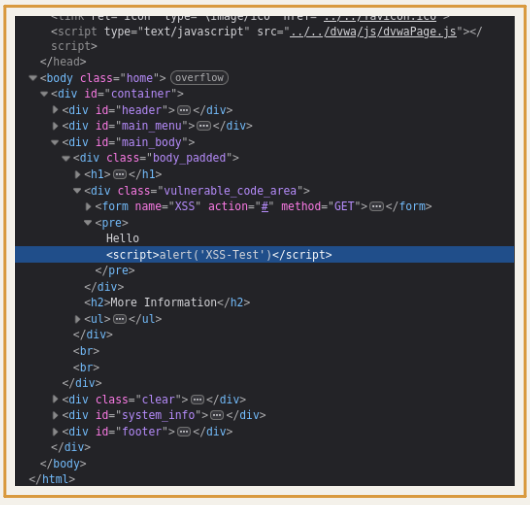


Fig 2.4 — Page source: unescaped <script> tag inside <pre>

DVWA DOM XSS — Marker Fired

The marker was placed directly in the **default** URL parameter. The dropdown UI visibly broke ("and press Enter." appeared as an option label) as the injected markup interfered with the intended HTML structure, and the alert fired. The live DOM Inspector shows the payload as an encoded artifact — this is expected, since the snapshot was taken after the browser had already re-parsed the page following execution.

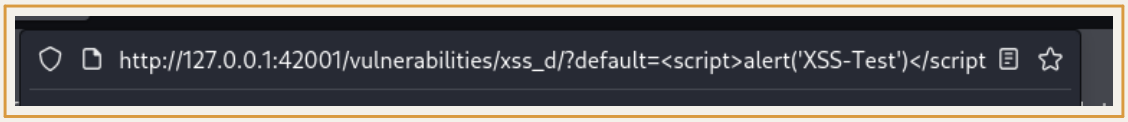


Fig 2.5 — Payload placed directly in default= URL parameter

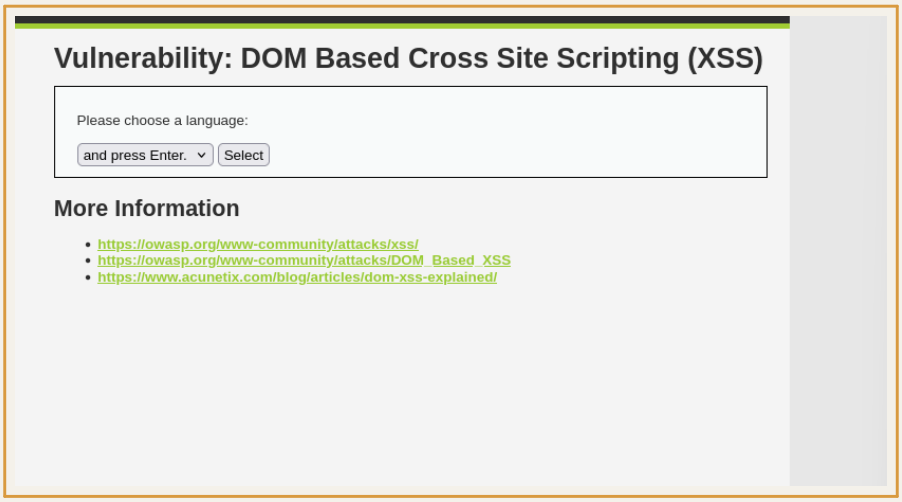


Fig 2.6 — Malformed dropdown UI confirming injection broke structure



Fig 2.7 — Alert executed: "XSS-Test"

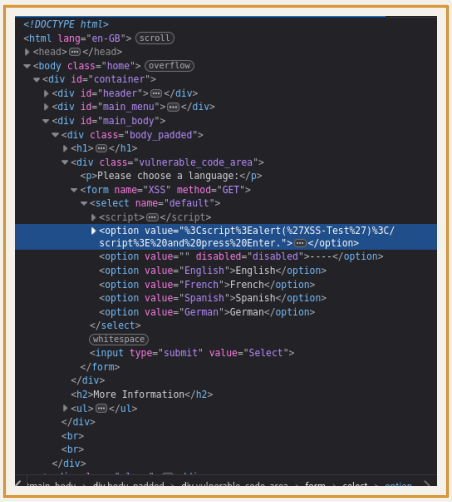


Fig 2.8 — Live DOM Inspector: encoded post-execution artifact in option value

Mutillidae Reflected XSS (DNS Lookup) — Marker Fired

The marker was submitted through the Hostname/IP field. The alert fired immediately, confirming the target_host parameter identified in Phase 1 is directly exploitable at the default security level.



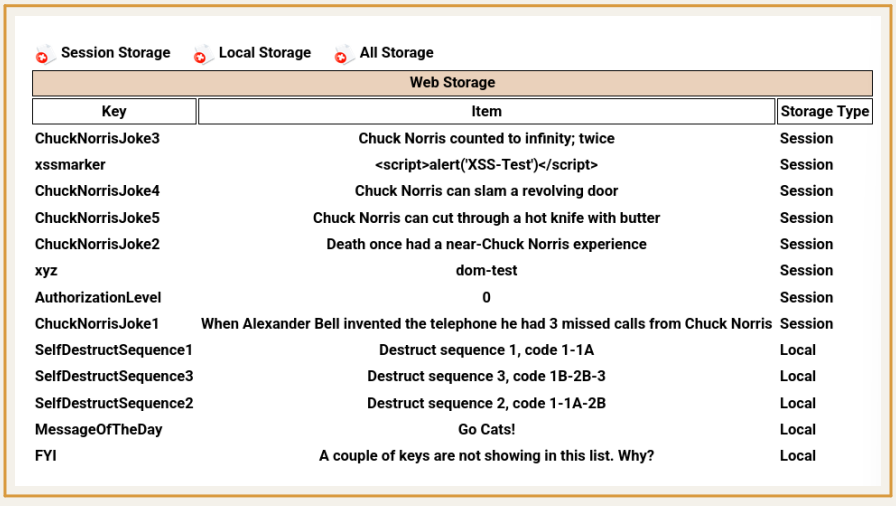
Fig 2.9 — Alert executed: "XSS-Test"



Fig 2.10 — Alert with target_host field visible in Inspector

Mutillidae DOM XSS (HTML5 Storage) — Marker Did NOT Execute (Negative Finding)

Unlike the other three vectors, this field renders stored values via a safe text-insertion method rather than innerHTML/document.write. The payload was stored correctly but displayed as inert literal text — no alert fired, even after a page refresh. This is a genuine negative result: not every client-storage-backed feature is automatically exploitable.



Session Storage | Local Storage | All Storage

Web Storage		
Key	**Item**	**Storage Type**
ChuckNorrisJoke3	Chuck Norris counted to infinity; twice	Session
xssmarker	<script>alert('XSS-Test')</script>	Session
ChuckNorrisJoke4	Chuck Norris can slam a revolving door	Session
ChuckNorrisJoke5	Chuck Norris can cut through a hot knife with butter	Session
ChuckNorrisJoke2	Death once had a near-Chuck Norris experience	Session
xyz	dom-test	Session
AuthorizationLevel	0	Session
ChuckNorrisJoke1	When Alexander Bell invented the telephone he had 3 missed calls from Chuck Norris	Session
SelfDestructSequence1	Destruct sequence 1, code 1-1A	Local
SelfDestructSequence3	Destruct sequence 3, code 1B-2B-3	Local
SelfDestructSequence2	Destruct sequence 2, code 1-1A-2B	Local
MessageOfTheDay	Go Cats!	Local
FYI	A couple of keys are not showing in this list. Why?	Local

Fig 2.11 — Payload stored but displayed as literal text, no execution



Fig 2.12 — Inspector confirming script tag sits as inert text in <td>

4. Phase 3 — Stored XSS Cross-Session Persistence Proof

The defining trait of stored XSS is that the payload executes for any session that later loads the affected page — not only the session that submitted it. This was proven by submitting the marker in a primary browser session, then loading the same page in a separate, independent private/incognito session, confirming execution without any resubmission.

DVWA Stored XSS

The marker was submitted fresh in an incognito window; the alert fired there immediately. Switching back to the original session and simply reloading the page (no resubmission) triggered the same alert again, confirming the payload is served identically to any session that loads it.

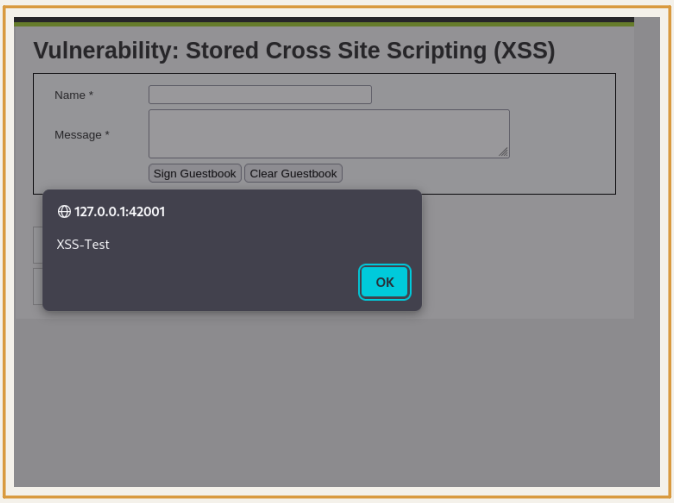


Fig 3.1 — Alert fires in incognito session after submission

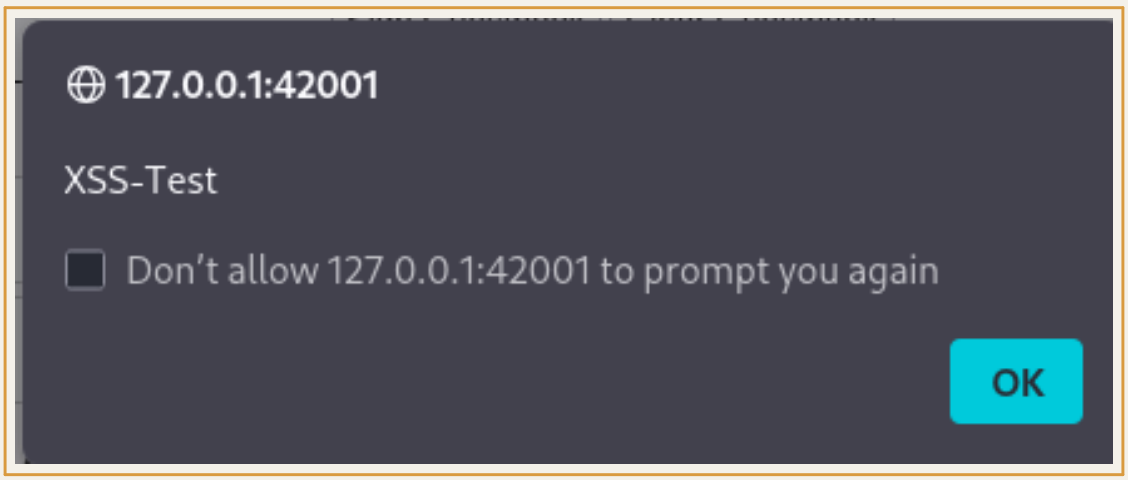


Fig 3.2 — Alert fires again in main session on simple reload

Mutillidae Stored XSS

An initial attempt using a single-quote payload (alert('XSS-Test')) caused a raw backend MySQL syntax error, an incidental finding indicating this field is also vulnerable to SQL injection (not exploited further, as SQL injection sits outside this lab's assessment scope). A double-quote variant was used instead, which fired cleanly in the submitting session and again in the second session, confirming the same cross-session persistence as DVWA.



Fig 3.3 — Incidental SQL syntax error from single-quote payload



Fig 3.4 — Alert fires in main session (double-quote payload)

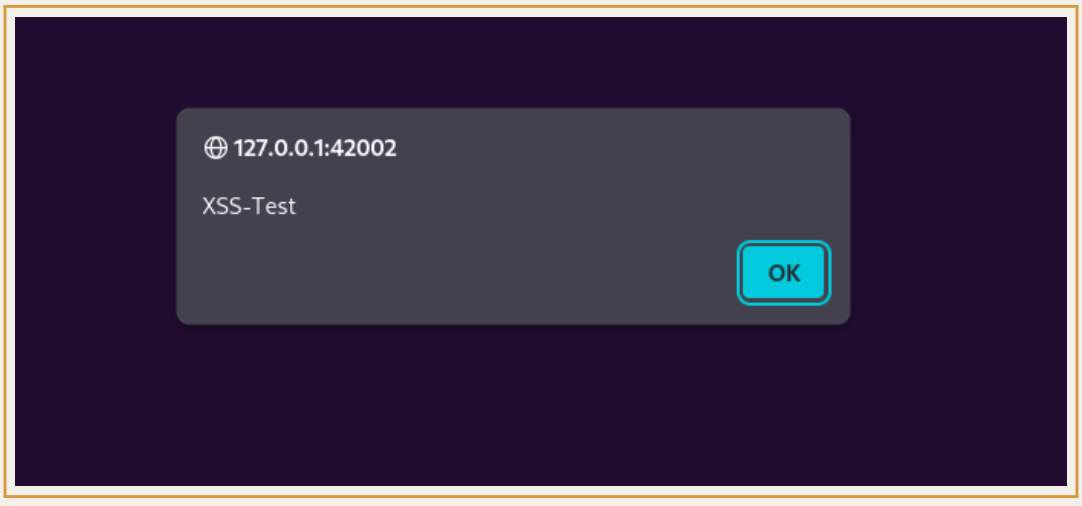


Fig 3.5 — Alert fires in second session confirming persistence

5. Phase 4 — XSS Secure-Mode Retest

Each application's security level was raised (DVWA: High; Mutillidae: raised via Toggle Security) and the same marker payloads were retried to characterise the encoding/sanitisation change, if any.

DVWA — Security Level: High

Both Reflected and Stored XSS were blocked at High: the <script> tag was stripped entirely by DVWA's filter, leaving only the inert text portion of the payload. The DOM XSS module, however, did not fire on retest — yet page-source inspection confirmed the underlying document.write() sink code is byte-for-byte identical between Low and High security levels. Non-execution is therefore attributed to browser-side navigation/caching behaviour rather than any real server-side defence, an important distinction for accurate reporting.

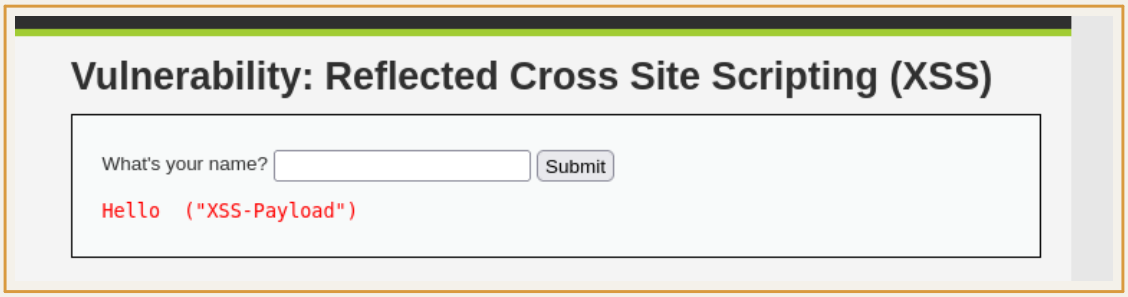


Fig 4.1 — Reflected XSS: <script> tag stripped, only text remained

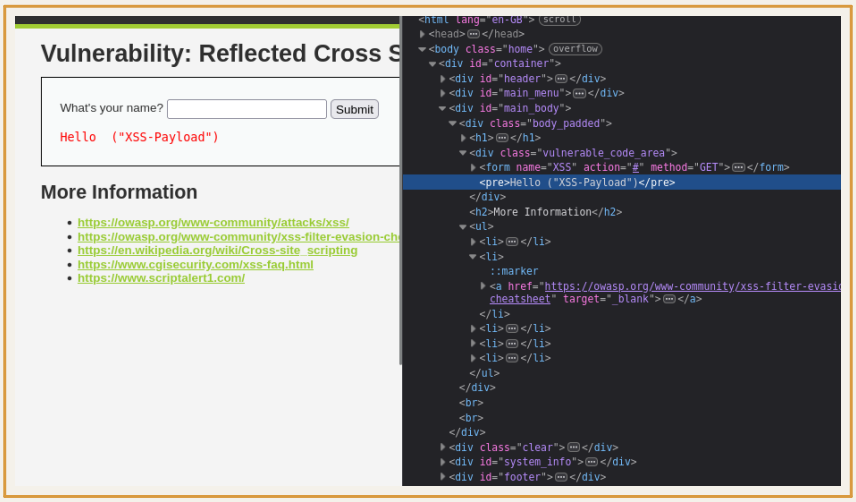


Fig 4.2 — Page source confirming tag removal

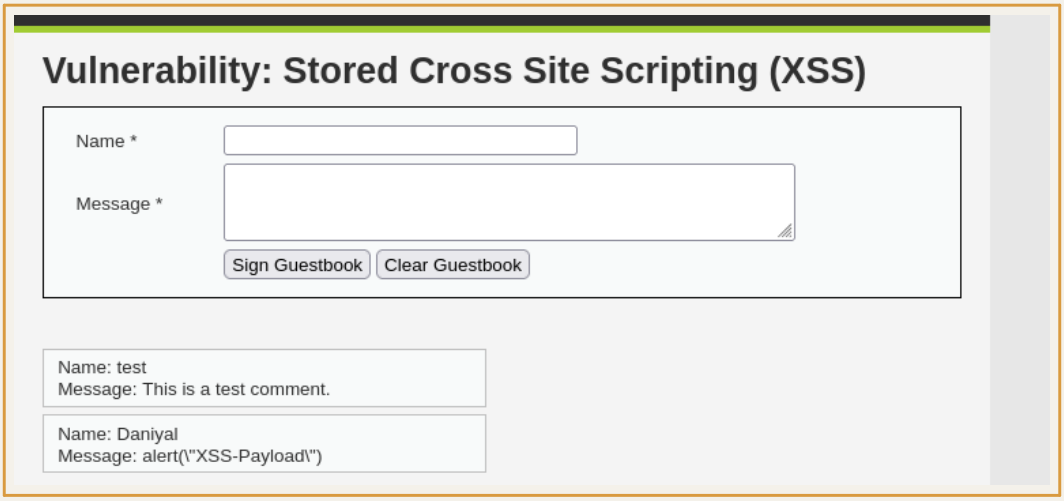


Fig 4.3 — Stored XSS: same stripping behaviour applied

```
<form name="XSS" method="GET">
    <select name="default">
        <script>
            if (document.location.href.indexOf("default=") >= 0) {
                var lang = document.location.href.substring(document.location.href.indexOf("default=")+8);
                document.write("<option value='" + lang + "'>" + decodeURI(lang) + "</option>");
                document.write("<option value='' disabled='disabled'>----</option>");
            }

            document.write("<option value='English'>English</option>");
            document.write("<option value='French'>French</option>");
            document.write("<option value='Spanish'>Spanish</option>");
            document.write("<option value='German'>German</option>");
        </script>
    </select>
    <input type="submit" value="Select" />
</form>
</div>
<h2>More Information</h2>
```

Fig 4.4 — DOM XSS source code, unchanged from Low-security version

Mutillidae — Raised Security Level

Stored XSS was blocked with genuine output encoding — the payload rendered in the blog table as an HTML-escaped literal string with backslash-escaped quotes, confirming real server-side sanitisation. Reflected (DNS Lookup) and DOM (HTML5 Storage) retests were inconclusive: both fields enforce a client-side character limit of approximately 20 characters, truncating the payload before valid <script>...</script> syntax could form — an incidental input-length constraint, not a designed XSS defence, and independent of the security-level toggle.

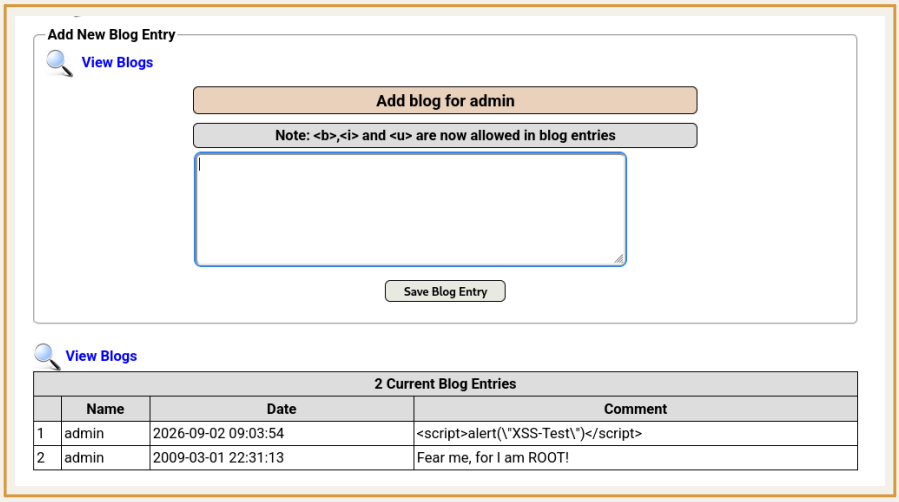


Fig 4.5 — Stored XSS blocked: payload HTML-escaped in blog output

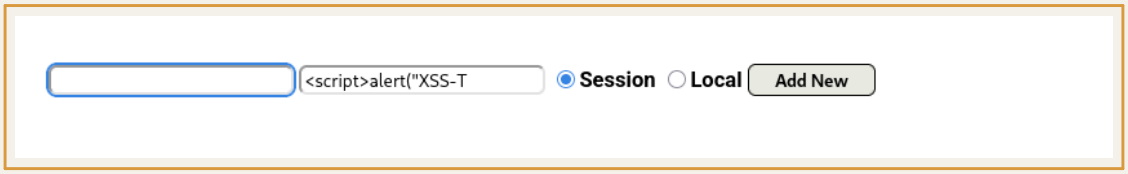


Fig 4.6 — HTML5 Storage field truncation

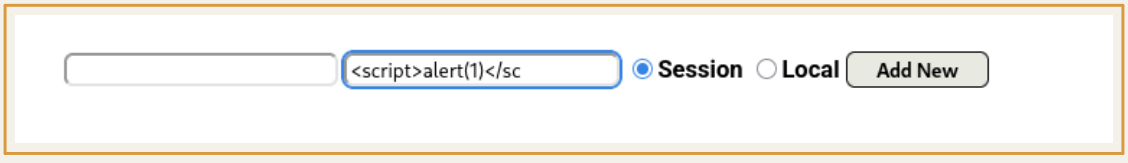


Fig 4.7 — Shorter payload, still truncated

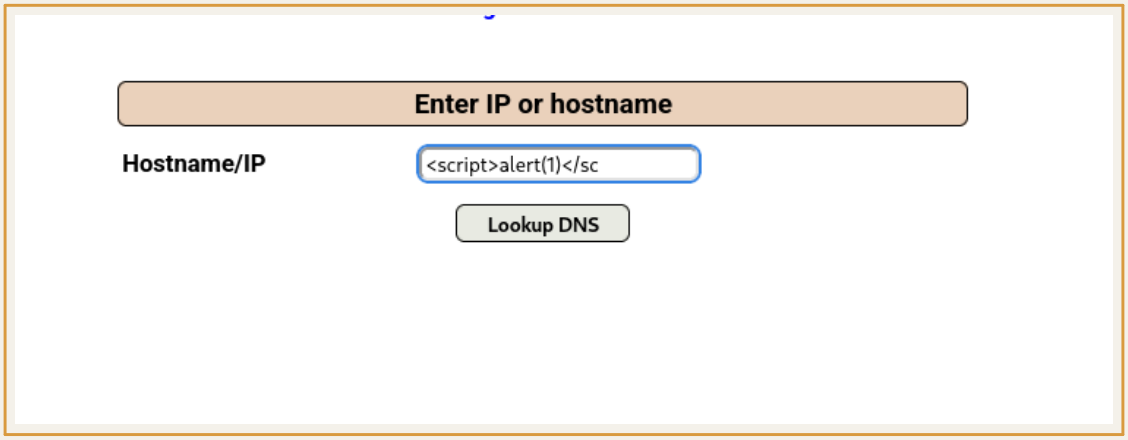


Fig 4.8 — DNS Lookup field, same truncation limit

6. Phase 5 — Session Lifecycle

Session cookies were captured at three points — pre-login, post-login and post-logout — to determine whether the session identifier rotates at login and whether the pre-logout session is genuinely invalidated server-side.

DVWA

The PHPSESSID value changed between the pre-login and post-login captures (ddb62fb7... to b54114a2...), confirming DVWA rotates the session identifier on successful authentication. The cookie value stayed the same after logout, but attempting to access a protected page directly afterward redirected to the login page, confirming the server-side session itself was correctly invalidated even though the client-side cookie value was left unchanged.

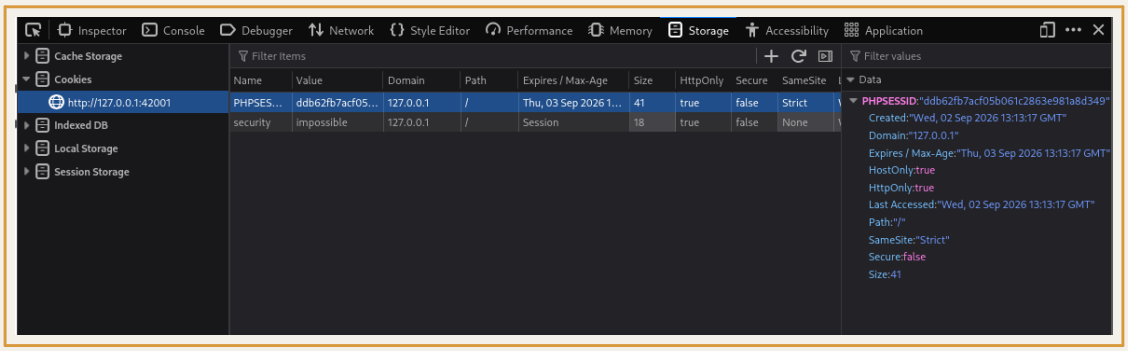


Fig 5.1 — Pre-login PHPSESSID: ddb62fb7...

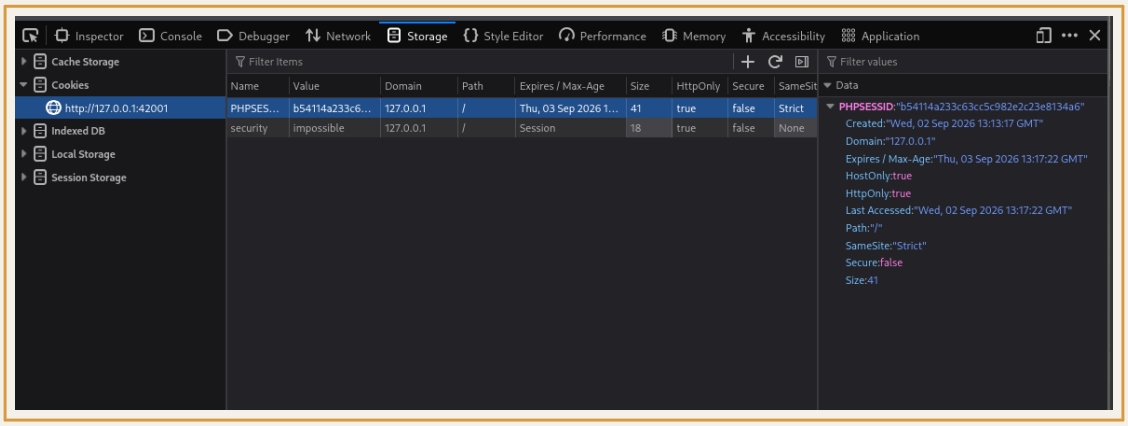


Fig 5.2 — Post-login PHPSESSID: b54114a2... (rotated)

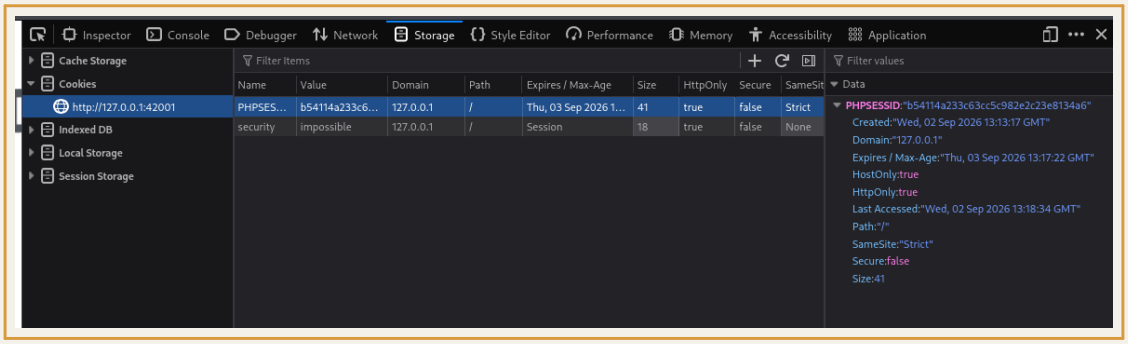


Fig 5.3 — Post-logout PHPSESSID: unchanged value, but server-side session invalidated

Mutillidae

By contrast, Mutillidae's PHPSESSID (4fc19cefe1e3...) was identical across all three captures — pre-login, post-login and post-logout — meaning the session identifier is never rotated at login, a genuine session-fixation vulnerability. Two additional cookies, **uid** and **username**, appeared after login, with the username stored in plaintext. As with DVWA, direct access to a protected page after logout correctly redirected to the login screen, confirming server-side invalidation still functions despite the lack of cookie rotation.

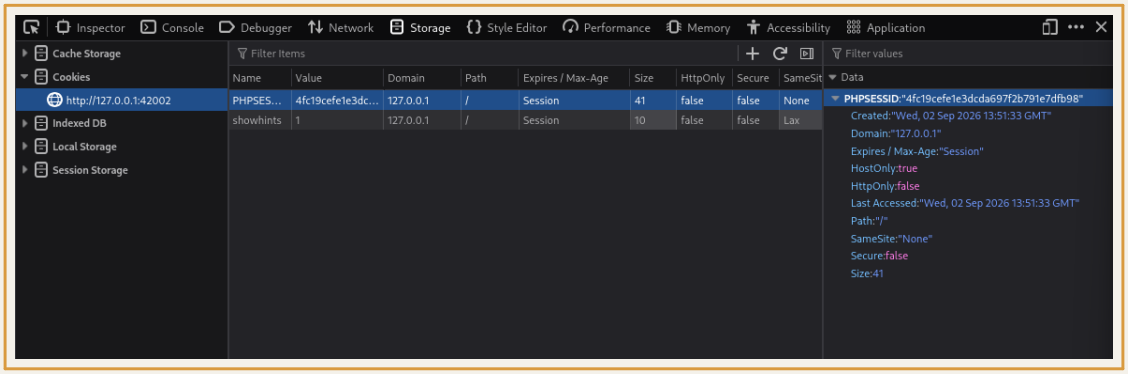


Fig 5.4 — Pre-login PHPSESSID: 4fc19cefe1e3...

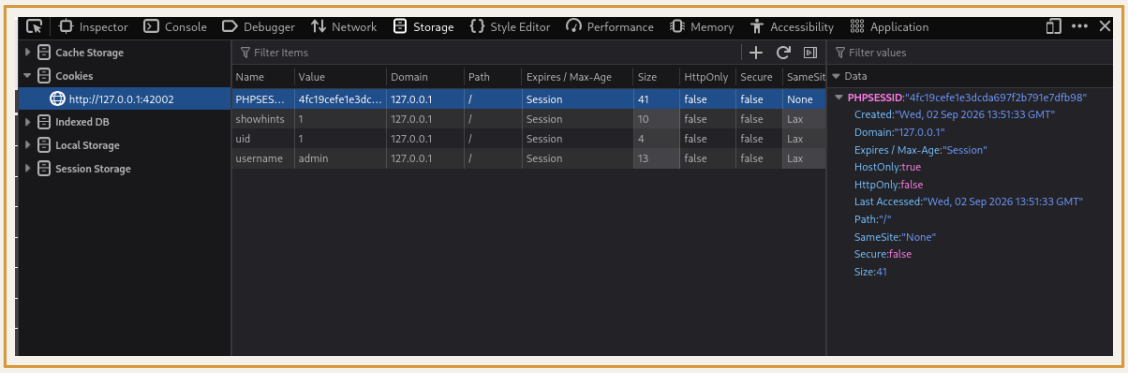


Fig 5.5 — Post-login: same PHPSESSID (no rotation); new uid/username cookies appear

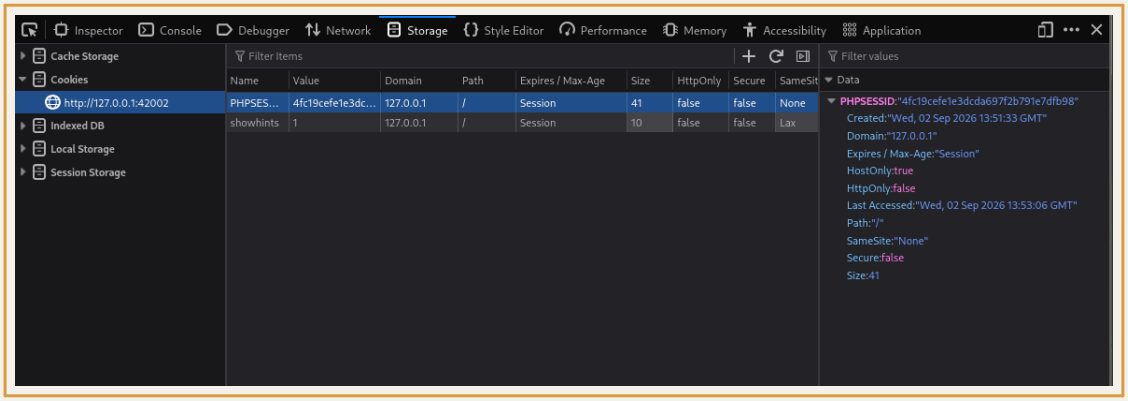


Fig 5.6 — Post-logout: PHPSESSID still unchanged

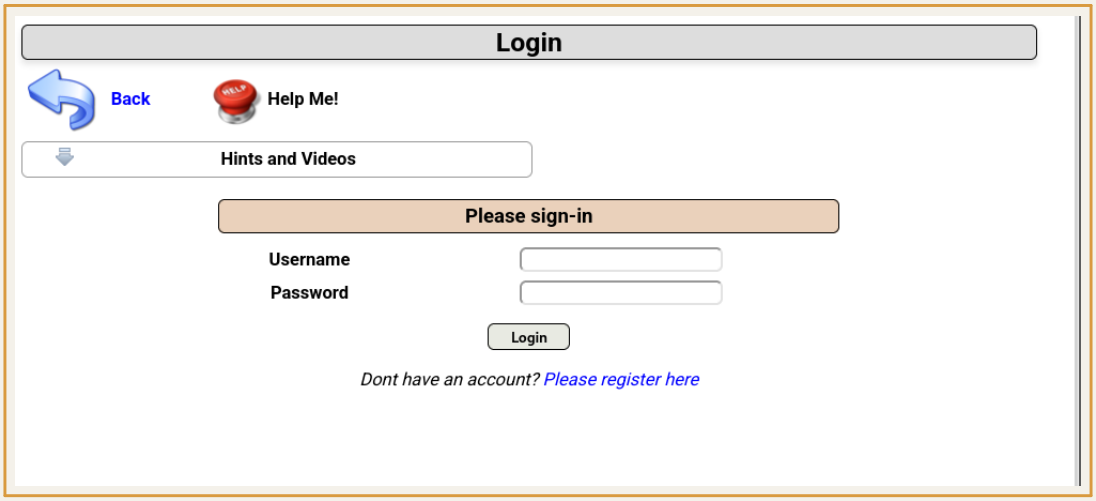


Fig 5.7 — Direct access to protected page after logout redirects to Login

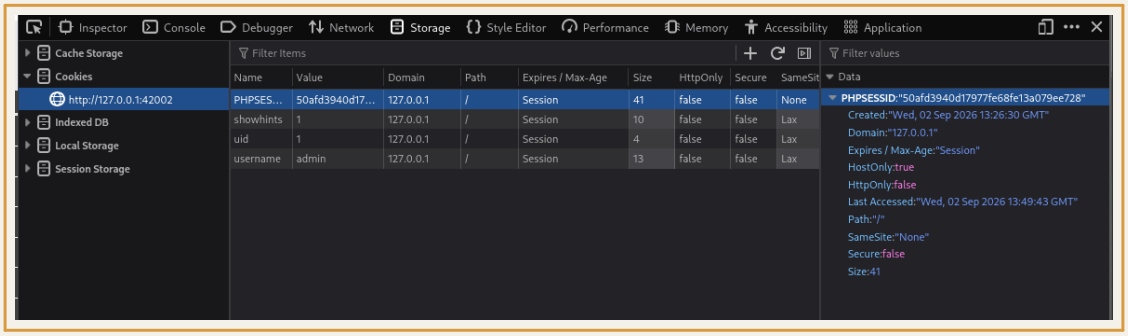


Fig 5.8 — Supplementary Mutillidae cookie capture, admin session, showing uid/username pattern

Finding: DVWA rotates its session identifier upon successful login, a defence against session-fixation attacks. Mutillidae does not — the identical PHPSESSID before and after authentication means an attacker who can set or predict a victim's pre-login session ID could hijack the account once the victim authenticates. Both applications correctly invalidate the session server-side on logout, even though neither clears or rotates the client-side cookie value at that point — a minor hygiene gap in both.

7. Phase 6 — Cookie Attribute Table

Cookie attributes captured directly from the browser's storage inspector during Phase 5 are consolidated below.

App	Cookie	Secure	HttpOnly	SameSite	Domain	Path	Expiry
DVWA	PHPSESSID	false	true	Strict	127.0.0.1	/	Session
DVWA	security	false	true	None	127.0.0.1	/	Session
Mutillidae	PHPSESSID	false	false	None	127.0.0.1	/	Session
Mutillidae	showhints	false	false	Lax	127.0.0.1	/	Session
Mutillidae	uid	false	false	Lax	127.0.0.1	/	Session
Mutillidae	username	false	false	Lax	127.0.0.1	/	Session

Critical cross-cutting finding: Mutillidae's session cookie is **HttpOnly: false**, and Phase 2/3 confirmed working stored XSS on the same application. This combination represents a realistic session-hijacking chain (stored payload → document.cookie read → session theft) — actual cookie theft was correctly excluded from this lab's permitted scope, but the chain is worth flagging explicitly in remediation. DVWA's **HttpOnly: true** setting blocks this exact chain even where its own XSS vulnerabilities exist at Low security. Both applications run on plain HTTP with **Secure: false**, which is expected in this local test environment but must never be carried into a production deployment.

8. Phase 7 — Authentication Resilience (5-Attempt Test)

A maximum of five failed login attempts was run against one dedicated, non-administrative account per target (DVWA: gordonb; Mutillidae: dani, self-registered for this test), followed by one correct-password login to confirm account availability, in strict compliance with the lab's five-attempt cap.

DVWA (gordonb)

Five consecutive wrong-password attempts all returned the same generic "Login failed" message with no perceptible delay, CAPTCHA, or lockout warning. The correct password on the sixth attempt logged in immediately, confirming the account was never actually locked.



Fig 7.1 — Attempt 1: "Login failed" (generic message)



Fig 7.2 — Subsequent failed attempt, same generic message, no delay



Fig 7.3 — Further failed attempt, no lockout/rate-limit indicator

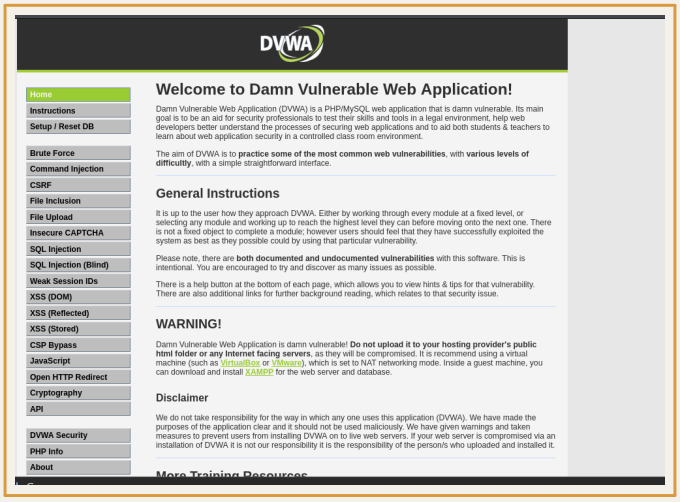


Fig 7.4 — Correct password: immediate successful login, no residual lockout

Mutillidae (dani)

The same five-attempt pattern was repeated against a self-registered Mutillidae account. Every failed attempt returned a username-specific "Password incorrect" message, again with no delay or lockout behaviour. The correct password immediately produced a successful "User Authenticated" status.

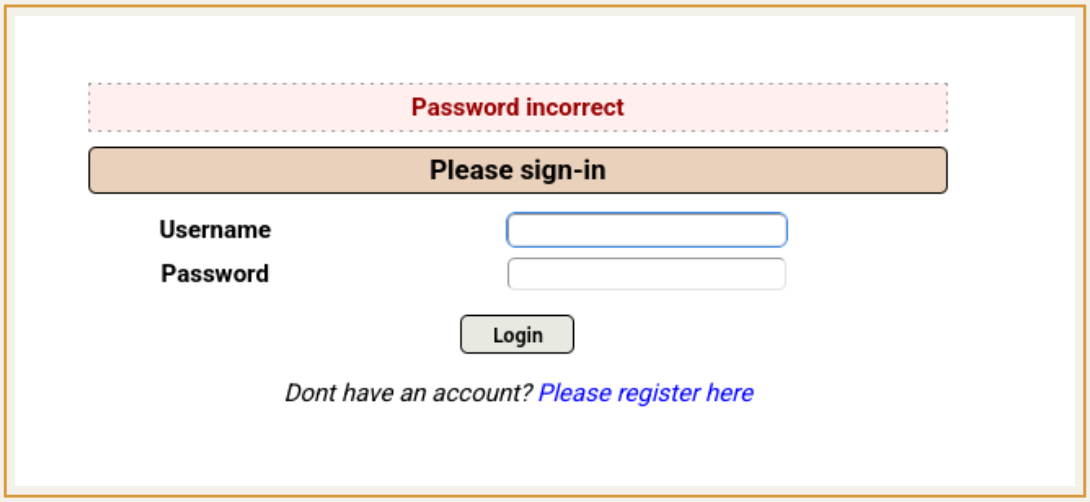


Fig 7.5 — Failed attempt: "Password incorrect" (username-specific message)



Fig 7.6 — Repeated failed attempt, same specific message



Fig 7.7 — Final failed attempt (5th), no delay or lockout indicator



Fig 7.8 — Correct password: successful login, "User Authenticated"

Target	Attempts 1-5	6th (correct)	Rate-limit / lockout observed?
DVWA	5x "Login failed", no delay	Successful, immediate access	None
Mutillidae	5x "Password incorrect", no delay	Successful, "User Authenticated"	None

Findings: Neither application implements rate-limiting, CAPTCHA or account lockout after repeated failed attempts. Mutillidae's specific "Password incorrect" wording is a username-enumeration risk not present in DVWA's generic "Login failed" message. Both accounts remained fully functional immediately after the test — no availability impact resulted from the controlled attempt sequence.

9. Phase 8 — Comparative Analysis

The table below consolidates every control area assessed across Phases 1–7 into a direct side-by-side comparison of the two target applications.

Control Area	DVWA	Mutillidae
Reflected XSS (low security)	Vulnerable	Vulnerable
Stored XSS (low security)	Vulnerable, persists cross-session	Vulnerable, persists cross-session
DOM XSS (low security)	Vulnerable	Not vulnerable (safe text render)
XSS at raised security	Reflected/Stored blocked; DOM unaffected by setting	Stored blocked (encoded); Reflected/DOM inconclusive (field length limits)
Session rotation at login	Yes	No — session fixation risk
Session invalidation at logout	Yes (server-side)	Yes (server-side)
Session cookie HttpOnly	true	false
Login error specificity	Generic ("Login failed")	Specific ("Password incorrect") — enumeration risk
Brute-force protection	None observed	None observed

Synthesis: DVWA and Mutillidae exhibit complementary weaknesses. DVWA's session handling is stronger (login-time rotation, HttpOnly cookie), but its DOM XSS module remains exploitable regardless of the configured security level, since the vulnerable code itself never changes. Mutillidae's output encoding at raised security is genuinely effective against stored XSS, but its session management carries a real fixation vulnerability, and its non-HttpOnly session cookie combined with the confirmed stored-XSS capability at Level 0 forms a realistic payload-to-cookie-theft attack chain — correctly not exploited further under this lab's scope, but an important finding for remediation prioritisation.

10. Analysis Questions

Q: What distinguishes reflected, stored and DOM XSS?

Reflected XSS occurs when the server echoes attacker-supplied input directly back into the immediate HTTP response, requiring the victim to click a crafted link or submit a crafted request each time. Stored XSS occurs when the payload is saved server-side (e.g. in a database) and served to any subsequent visitor who loads the affected page, without needing to resubmit anything — as demonstrated in Phase 3, where the payload executed in a completely independent second browser session. DOM XSS differs from both in that the payload is never processed by the server at all; client-side JavaScript reads an attacker-controlled source (such as document.location) and writes it into the page via an unsafe sink such as document.write or innerHTML, meaning server-side filtering alone cannot prevent it — confirmed directly in Phase 4, where DVWA's High security setting left the DOM XSS module's source code completely unchanged.

Q: Why is a harmless visual marker enough to prove browser execution?

A visible, unmistakable effect — such as a JavaScript alert() dialog displaying a unique string — is definitive proof that the browser parsed and executed attacker-controlled input as active script rather than treating it as inert text. It requires no destructive payload, no data exfiltration and leaves no residual effect on the target application once dismissed, satisfying both the technical proof-of-concept requirement and the lab's harmless-testing constraint simultaneously.

Q: Why should a session rotate at login and become invalid after logout?

Rotating the session identifier at login defeats session-fixation attacks, where an attacker pre-sets or predicts a victim's session ID before authentication and then reuses that same ID after the victim logs in to inherit their authenticated session. Phase 5 demonstrated this concretely: DVWA issues a new PHPSESSID on login (fixation-resistant), while Mutillidae keeps the identical PHPSESSID throughout (fixation-vulnerable). Invalidating the session on logout ensures that a stolen or leaked cookie cannot be reused indefinitely after the legitimate user has ended their session — both applications correctly enforced this server-side, even though neither rotated or cleared the client-side cookie value itself at logout.

Q: How can account lockout itself create availability risk?

An aggressive lockout policy can be weaponised by an attacker as a denial-of-service vector: by deliberately submitting repeated failed login attempts against a known username, an attacker can lock a legitimate user out of their own account without ever needing to guess the correct password. This is why modern authentication design favours graduated defences — such as IP-based rate-limiting, exponential backoff delays, or CAPTCHA challenges — over a simple hard lockout counter, balancing brute-force resistance against this self-inflicted availability risk. Neither DVWA nor Mutillidae implemented any lockout mechanism in this assessment, so this specific tradeoff was not directly observable in practice here, but is an important design consideration for the remediation recommendations.

11. Remediation Table

Finding	Recommendation
Reflected/Stored/DOM XSS (low security, both apps)	Context-aware output encoding (HTML-entity encode for HTML context, JS-string escape for script context); use safe DOM APIs (textContent, not innerHTML/document.write).
DVWA DOM XSS unaffected by security level	Sanitise/encode the 'default' parameter server-side before it ever reaches client-side script; do not rely on a security-level toggle for DOM-sink vulnerabilities.
Mutillidae session fixation (no rotation at login)	Regenerate the session identifier on every privilege change — login and logout — e.g. session_regenerate_id(true) in PHP.
Mutillidae session cookie HttpOnly: false	Set HttpOnly: true on all session cookies to block JavaScript-based cookie theft even where a stored-XSS vulnerability exists.
Cookies unchanged (not rotated) at logout, both apps	Regenerate or explicitly clear and reissue the session cookie on logout, not only the server-side session state.
Secure: false on all cookies, both apps	Enforce HTTPS in production and set the Secure flag so cookies are never transmitted over plaintext HTTP.
No brute-force protection on either login form	Add rate-limiting (e.g. exponential backoff per IP/account) and/or temporary lockout after N failed attempts, balanced against lockout-based availability risk (e.g. throttle by IP rather than permanently locking the account).
Mutillidae's specific "Password incorrect" message	Use a single generic error message for both invalid username and invalid password to prevent username enumeration.
No MFA on either application	Add multi-factor authentication (e.g. TOTP) as a defence-in-depth layer independent of password strength.

12. Evidence Register

The full evidence register (63 screenshots across Phases 1-7, each numbered and mapped to its source application, action, output and purpose) is maintained in the accompanying evidence_log.md file included in the GitHub submission package. A condensed phase-level summary is provided below.

Phase	Source App(s)	Action	Count	Purpose
1	DVWA + Mutillidae	Baseline input/sink mapping	22	Establish parameter, method, sink for each of 6 XSS vectors
2	DVWA + Mutillidae	Harmless marker injection	12	Prove browser-side script execution
3	DVWA + Mutillidae	Cross-session reload	5	Prove stored-XSS persistence across sessions
4	DVWA + Mutillidae	Secure-mode retest	8	Characterise encoding/sanitisation change
5	DVWA + Mutillidae	Cookie capture (3 points)	8	Determine rotation and invalidation behaviour
6	DVWA + Mutillidae	(derived from Phase 5)	0	Cookie attribute documentation
7	DVWA + Mutillidae	5-attempt auth test	8	Assess rate-limit/lockout behaviour

13. Evidence-Minimisation Statement

All evidence captured for this assessment consists solely of harmless proof-of-concept markers (a fixed JavaScript alert string), screenshots of the learner's own dedicated test-account sessions, and non-sensitive session-cookie metadata (names, flags and values belonging only to the learner's own local, isolated VM instances). No real user data, third-party credentials, or production system information was accessed, captured or retained at any point. Session tokens visible in this report belong exclusively to locally hosted, non-networked training applications and were rotated/invalidated as part of the Phase 9 cleanup, rendering them non-functional beyond this assessment.

14. Cleanup & Restoration Checklist

- Stored XSS markers removed/reset in both DVWA (guestbook cleared) and Mutillidae (blog entries reset).
- HTML5 Storage test entries (Session/Local) cleared in Mutillidae.
- All test sessions logged out, including incognito/private windows used for cross-session testing.
- Dedicated browser profiles and cookies cleared for both target applications.
- Test accounts (gordonb, dani) confirmed not locked out following the 5-attempt authentication test.
- Evidence reviewed to confirm no live credentials, full session tokens, or wordlists are exposed in the submission package outside the protected evidence directory.

Cleanup actions were confirmed completed by the learner directly in the VM; screenshots were not captured for this administrative phase, as the actions are non-evidentiary housekeeping steps rather than assessment findings.

15. Conclusion

This assessment confirmed exploitable Reflected, Stored and DOM-based XSS in both DVWA and Mutillidae at low security settings, with harmless-marker proofs captured for each vector and cross-session persistence formally demonstrated for both applications' stored XSS. Raising each application's security level meaningfully mitigated Reflected and Stored XSS through output encoding, but DVWA's DOM XSS module was found to be entirely unaffected by its security-level setting, since the underlying client-side sink code does not change between levels — a finding that underscores why DOM XSS requires dedicated client-side remediation rather than reliance on server-side filtering alone. Session-management testing revealed a meaningful asymmetry between the two applications: DVWA rotates its session identifier at login and marks its session cookie HttpOnly, while Mutillidae does neither, creating both a session-fixation exposure and a cookie-theft pathway that compounds its own confirmed XSS weaknesses. Authentication-resilience testing found no rate-limiting or lockout protection on either application, alongside a username-enumeration risk unique to Mutillidae's login error messaging. The remediation table in Section 11 provides layered, prioritised recommendations addressing each of these findings, spanning input/output encoding, safe DOM API usage, session-identifier hygiene, secure cookie attributes, and authentication rate-limiting with multi-factor authentication as defence in depth.